



Algoritmos e Estrutura de Dados I
Anotações de Aula
Profa Melise Maria Veiga de Paula

Matriz com Alocação Dinâmica

Em C, quando criamos uma matriz dinâmica usando ponteiro de ponteiro (int **), o processo ocorre em duas etapas:

- 1- Alocar um vetor de ponteiros para representar as linhas:

```
int **m = (int **) malloc(quantidadelinhas * sizeof(int*));
```

- Cada elemento m[i] é um ponteiro que vai apontar para uma linha.
- A variável m está alocada na Stack
- Já o vetor de ponteiros está alocado na heap
- O tamanho deste vetor é igual a quantidade de linhas

- 2- Alocar cada linha separadamente:

```
for (int i = 0; i < linhas; i++) {  
    m[i] = malloc(colunas * sizeof(int));  
}
```

- Agora, cada m[i] aponta para um bloco de inteiros na heap.
- m[i] é um ponteiro e guarda o endereço do primeiro elemento da linha i
- Os elementos de cada linha são alocados de forma contígua na heap
- Linhas diferentes não são alocadas necessariamente de forma contígua como em uma matriz alocada de forma estática

A seguir, será apresentado um roteiro para a elaboração de um programa que vai permitir que você veja isso na prática.

Elabore um programa em C que execute as seguintes ações:

- a) Leia os valores de l e c que representam, respectivamente, a quantidade de linhas e colunas de uma matriz de inteiros
- b) Aloque a matriz dinamicamente, primeiro aloque um vetor de ponteiros de inteiros de tamanho l. Cada elemento desse vetor é um (int *) (passo 1)
- c) Depois faça cada elemento do vetor alocado apontar para um vetor de inteiros alocado dinamicamente de maneira que os valores dos elementos da linha i sejam armazenados no vetor apontado por m[i] (passo 2)
- d) Preencher a matriz com valores simples digitados pelo usuário
- e) Imprimir a matriz linha por linha
- f) Para cada linha, imprima o endereço de cada elemento (linha por linha) - &m[i][j]



MINISTÈRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

g) Para i variando de 0 até a quantidade de linhas -1, imprima o valor de $m[i]$

Responda as seguintes perguntas:

1. O endereço de $m[i]$ é igual a $\&m[i][0]$?
2. Os elementos $m[i][j]$ de uma mesma linha são contíguos?
3. Os endereços dos elementos de linhas diferentes são contíguos? Por exemplo, o endereço do último elemento da linha 0 é seguido do endereço do primeiro elemento da linha 1?



MINISTÉRIO DA EDUCAÇÃO
UNIVERSIDADE FEDERAL DE ITAJUBÁ

Criada pela Lei nº 10.435 – 24/04/2002

Sugestão de implementação:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int linhas, colunas;
    printf("Digite linhas e colunas: ");
    scanf("%d %d", &linhas, &colunas);
    // 1) Alocação sem o teste dos ponteiros – é necessário complementar
    int **m = (int**) malloc(linhas * sizeof(int*));
    for (int i = 0; i < linhas; i++) {
        m[i] = (int*) malloc(colunas * sizeof(int));
    }
    // 2) Preenchimento com dados quaisquer não digitados pelo usuário
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++) {
            m[i][j] = i * 10 + j; // valor para fácil identificação
        }
    }
    // 3) Impressão dos valores
    printf("\nMatriz:\n");
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++) {
            printf("%d ", m[i][j]);
        }
        printf("\n");
    }
    // 4) Impressão dos endereços
    printf("\nEndereços:\n");
    for (int i = 0; i < linhas; i++) {
        for (int j = 0; j < colunas; j++) {
            printf("&m[%d][%d] = %p", i, j, &m[i][j]);
        }
        printf("\n");
        printf("m[%d] = %p | &m[%d][0] = %p\n", i, m[i], i, &m[i][0]);
        printf("\n");
    }
    // 5) Liberação
    for (int i = 0; i < linhas; i++) free(m[i]);
    free(m);
    return 0;
}
```